CUNEF UNIVERSIDAD

Relational databases · Databases practice

SQL: summary

1. **Oracle and FreeSQL**: Practising on Oracle with FreeSQL · Oracle compared with other SQL dialects
2. **Querying**: SELECT: columns, expressions and aliases · Sorting results with ORDER BY · Filtering with WHERE, AND, OR and NOT · BETWEEN, IN and LIKE · Working with NULL · Joining tables with INNER JOIN
3. **Combining tables**: Outer joins: LEFT, RIGHT and FULL · CROSS JOIN, self-joins, USING and NATURAL
4. **Aggregation**: Aggregate functions: COUNT, SUM, AVG, MIN and MAX · GROUP BY and HAVING
5. **Subqueries & sets**: Subqueries in WHERE, FROM and SELECT · Multi-row subqueries: IN, ANY and ALL · Correlated subqueries and EXISTS · Set operators: UNION, INTERSECT and MINUS
6. **Functions**: Text functions · Number and date functions · Conversion, NULL functions, CASE and DECODE
7. **SQL languages**: The sub-languages of SQL · DDL: defining the schema · CREATE TABLE step by step · ALTER, DROP and TRUNCATE · DML: reading and changing rows · INSERT in practice · UPDATE and DELETE in practice · DCL: users, roles and privileges · TCL: transactions
8. **Constraints**: PRIMARY KEY, UNIQUE and NOT NULL · FOREIGN KEY and referential actions · CHECK and DEFAULT
9. **Views & schema objects**: Views · WITH: common table expressions · Sequences and synonyms
10. **PL/SQL**: PL/SQL blocks, variables and SELECT INTO · Control flow: IF, CASE and loops · Cursors: walking through the rows one by one · Exceptions: handling errors · Stored procedures and functions · Triggers · Packages
11. **Indexes**: What an index is · Composite and covering indexes · What indexes cost · Kinds of index
12. **Clustering**: Clustered and non-clustered indexes · Table clusters in Oracle (and CLUSTER in PostgreSQL)
13. **Partitioning**: Horizontal partitioning · Partition pruning and maintenance · Vertical partitioning and sharding
14. **Efficiency**: Reading a query plan · Conditions that can use an index · Writing efficient queries

Oracle and FreeSQL

Practising on Oracle with FreeSQL

FreeSQL (freesql.com) is Oracle's free online worksheet: you write Oracle SQL in the browser and run it on a real Oracle database, with nothing to install. It is where you practise in this course.

- Run one statement: put the cursor in it and run. Run the whole script: use the script option of the worksheet.
- Your schema persists between sessions: drop what you no longer need.
- The simulation here does not support PL/SQL, partitioning, sequences or privileges: for those, FreeSQL is the only place to run them.

> **Common mistake.** Trusting only the simulation. It translates a subset of Oracle to a different engine: when its result surprises you, check it in FreeSQL.

Oracle compared with other SQL dialects

SQL is a standard, but every DBMS has its own dialect. Most errors of students who learned SQL elsewhere come from a few differences: row limiting, types, dates, empty strings and transactions.

Oracle versus SQLite, PostgreSQL and MySQL

Topic	Oracle	Others
First n rows	`FETCH FIRST n ROWS ONLY`	`LIMIT n`
Set difference	`MINUS`	`EXCEPT`
Text type	`VARCHAR2(n)`, `CLOB`	`TEXT`, `VARCHAR(n)`
Numbers	`NUMBER(p, s)`	`INTEGER`, `NUMERIC`, `REAL`
Auto number	`GENERATED ALWAYS AS IDENTITY`	`AUTOINCREMENT`, `SERIAL`, `AUTO_INCREMENT`
Current date	`SYSDATE`	`date('now')`, `CURRENT_DATE`, `NOW()`
Date literal	`DATE '2024-01-31'`, `TO_DATE(…)`	A plain string
Null replacement	`NVL(a, b)`, `COALESCE`	`IFNULL(a, b)`
Empty string	Is **NULL**	Is an empty string
Select without table	`SELECT 1 FROM dual`	`SELECT 1`
Transactions	Start with the first DML; DDL commits	`BEGIN` / `START TRANSACTION`
Data dictionary	`USER_TABLES`, `USER_TAB_COLUMNS`	`sqlite_master`, `information_schema`

Topic	Oracle	Others
Execution plan	`EXPLAIN PLAN FOR` + `DBMS_XPLAN`	`EXPLAIN`

> **Common mistake.** Writing a comparison like `WHERE signup_date >= '2024-01-01'` against a DATE column. It relies on the session's date format and often fails (ORA-01861); write `DATE '2024-01-01'`.

Querying

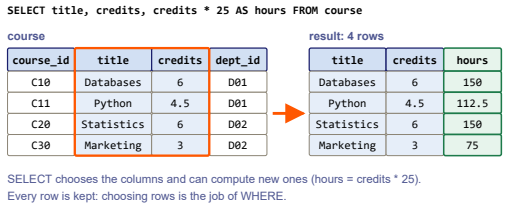
SELECT: columns, expressions and aliases

A SELECT names the columns you want to see, FROM names the table they come from. Each column can be a stored value, a calculation or a piece of glued text, and an alias gives it a readable name.

- Order of a basic query: `SELECT columns FROM table`; the clauses you will add later (WHERE, ORDER BY) come after FROM.
- Single quotes are for text values; double quotes are for names (aliases, columns).
- An alias only renames the output: it does not change the table.

Example you can run on the website

```
SELECT title, credits, credits * 25 AS hours
FROM course;
SELECT student_id || ' - ' || name AS student_label
FROM student;
SELECT DISTINCT dept_id
FROM course;
```



SELECT chooses the columns and can compute new ones; every row of the table stays.

> **Example.** `SELECT title, credits * 25 AS hours FROM course` shows each course with its study hours (1 ECTS credit ≈ 25 hours).

> **Common mistake.** Gluing text with + (`name + ' ' + email`). In Oracle + is only arithmetic and fails with ORA-01722 (invalid number): use `||`.

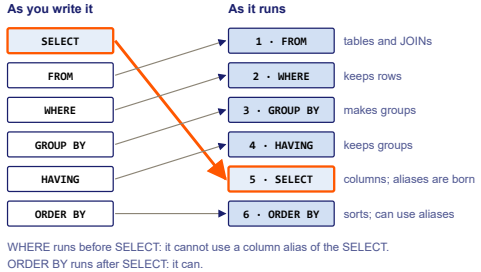
Sorting results with ORDER BY

Without ORDER BY a table has no guaranteed order: the rows come in whatever order the DBMS finds them. ORDER BY sorts the result by one or more keys, ascending (ASC, the default) or descending (DESC).

- ORDER BY is always the last clause of the query (FETCH FIRST goes after it).
- Text is sorted character by character: in Oracle's default binary sort, capital letters come before lower-case ones.
- If two rows tie on every key their order is not defined: add a key that makes it unique.

Example you can run on the website

```
SELECT title, credits
FROM course
ORDER BY credits DESC, title;
SELECT name, email
FROM student
ORDER BY email NULLS FIRST;
SELECT student_id, course_id, grade
FROM enrolment
ORDER BY grade DESC
FETCH FIRST 3 ROWS ONLY;
```



The order you write the clauses in is not the order they run in: SELECT comes fifth, so only ORDER BY can see its aliases.

> **Example.** `ORDER BY grade DESC FETCH FIRST 3 ROWS ONLY` gives the three best grades.

> **Common mistake.** Writing `DESC` once and expecting it to apply to every key. In `ORDER BY credits, title DESC` only the title is descending.

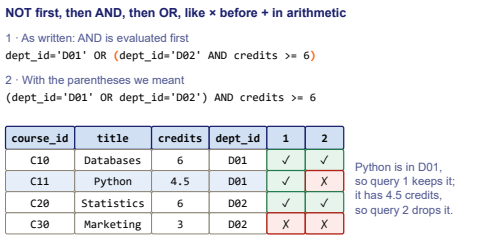
Filtering with WHERE, AND, OR and NOT

WHERE keeps only the rows for which a condition is true. Conditions compare values (=, <>, <, <=, >, >=) and combine with AND, OR and NOT.

- Text comparisons are case-sensitive in Oracle: `'spain'` is not `'Spain'`.
- Numbers are written without quotes: `credits >= 6`.
- Read your WHERE aloud with the parentheses: if the sentence is not what the question asks, the query is wrong.

Example you can run on the website

```
-- Without parentheses: AND is evaluated first
SELECT course_id, title, credits, dept_id
FROM course
WHERE dept_id = 'D01' OR dept_id = 'D02' AND credits >= 6;
-- With parentheses: what we actually meant
SELECT course_id, title, credits, dept_id
FROM course
WHERE (dept_id = 'D01' OR dept_id = 'D02') AND credits >= 6;
SELECT title
FROM course
WHERE NOT dept_id = 'D01';
```



The same three conditions, with and without parentheses. Only Python changes sides.

> **Example.** `WHERE (dept_id = 'D01' OR dept_id = 'D02') AND credits >= 6` keeps the 6-credit courses of either department.

> **Common mistake.** Leaving out the parentheses in a mix of AND and OR. `dept_id = 'D01' OR dept_id = 'D02' AND credits >= 6` returns **every** D01 course, whatever its credits.

BETWEEN, IN and LIKE

Three shortcuts for common filters: BETWEEN for ranges, IN for lists of values and LIKE for text patterns.

- `BETWEEN 2020 AND 2021` on a DATE column is wrong: compare with dates, `BETWEEN DATE '2020-01-01' AND DATE '2021-12-31'`.
- Watch out for spaces in patterns: `'The %'` (the word "The") is not `'The%'` (which also matches "Theo").
- The browser simulation ignores case in LIKE; Oracle does not. Check case-sensitive patterns on FreeSQL.

Example you can run on the website

```
SELECT student_id, course_id, grade
FROM enrolment
WHERE grade BETWEEN 7 AND 8.5;
SELECT course_id, title
FROM course
WHERE course_id IN ('C10', 'C30');
SELECT name
FROM student
WHERE name LIKE '%z';
```

> **Example.** `WHERE grade BETWEEN 7 AND 8.5` returns the grades 7.0, 7.5 and 8.5: both limits are included.

> **Common mistake.** Writing a pattern with =: `name = 'A%'` looks for a name that is literally "A%". Patterns need LIKE.

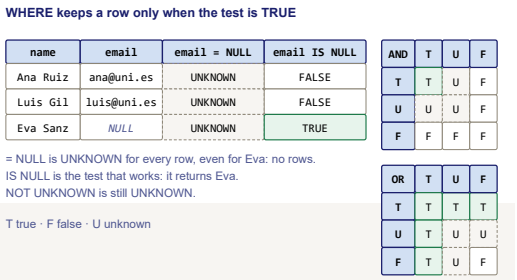
Working with NULL

NULL means "unknown" or "missing": it is not zero and not an empty text. Any comparison with NULL is unknown, so `= NULL` never finds anything: use IS NULL and IS NOT NULL.

- `NOT (email = NULL)` is still unknown: negating unknown does not make it true.
- A condition such as `grade <> 5` also skips the rows where grade is NULL.
- With ORDER BY, Oracle puts NULLs last in ascending order (see the ORDER BY card).

Example you can run on the website

```
-- Returns nothing: a comparison with NULL is never true
SELECT name, email FROM student WHERE email = NULL;
SELECT name FROM student WHERE email IS NULL;
SELECT name, NVL(email, 'no email') AS email FROM student;
```



Example. `SELECT name FROM student WHERE email IS NULL` returns the students without an email.

Common mistake. Writing `WHERE email = NULL` or `WHERE email = ''`: neither returns a row. Use `IS NULL`.

Comparing with NULL gives UNKNOWN, and WHERE drops UNKNOWN rows just like FALSE ones.

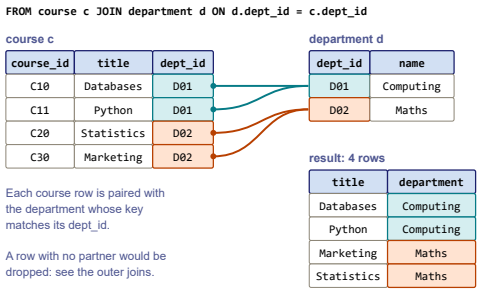
Joining tables with INNER JOIN

A JOIN combines rows of two tables that are related. The ON condition says how they match, almost always by following a foreign key to the primary key it references.

- One JOIN per foreign key you follow: n tables need n − 1 join conditions.
- WHERE and ORDER BY come after all the JOINs and can use columns of any joined table.
- Without the ON condition every row would be paired with every row of the other table (a Cartesian product).

Example you can run on the website

```
SELECT c.title, d.name AS department
FROM course c
JOIN department d ON d.dept_id = c.dept_id
ORDER BY d.name, c.title;
SELECT s.name, c.title, e.grade
FROM enrolment e
JOIN student s ON s.student_id = e.student_id
JOIN course  c ON c.course_id  = e.course_id
WHERE e.grade >= 7
ORDER BY s.name, c.title;
```



The ON condition follows the foreign key dept_id to the department it references.

Watch the video on the website, in the SQL section.

Example. `SELECT s.name, c.title FROM enrolment e JOIN student s ON s.student_id = e.student_id JOIN course c ON c.course_id = e.course_id` lists who is enrolled in what.

Common mistake. Joining on the wrong columns (`ON c.course_id = d.dept_id`). The query runs but the rows are nonsense or missing: the ON must follow a foreign key.

Combining tables

Outer joins: LEFT, RIGHT and FULL

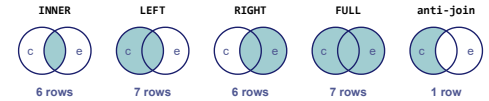
An OUTER JOIN also keeps the rows that find no partner, and fills the missing side with NULL. LEFT keeps every row of the left table, RIGHT every row of the right one, FULL every row of both.

- A condition on the right table goes in the **ON** if you want to keep the unmatched rows; in the WHERE it removes them again.
- To count partners, count a column of the right table (`COUNT(e.course_id)`): an unmatched course still produces one row, and `COUNT(*)` would count it as 1.
- Oracle's old `(+)` syntax (`WHERE c.course_id = e.course_id(+)`) is a LEFT JOIN: read it in old code, write the ANSI form.

Example you can run on the website

```
-- INNER: Marketing (no enrolments) disappears
SELECT c.title, e.student_id, e.grade
FROM course c
JOIN enrolment e ON e.course_id = c.course_id
ORDER BY c.title;
-- LEFT: every course, with NULL where nobody is enrolled
SELECT c.title, e.student_id, e.grade
```

```
FROM course c
LEFT JOIN enrolment e ON e.course_id = c.course_id
ORDER BY c.title;
-- Anti-join: the courses nobody takes
SELECT c.course_id, c.title
FROM course c
LEFT JOIN enrolment e ON e.course_id = c.course_id
WHERE e.course_id IS NULL;
-- FULL: who takes Statistics (C20), who takes Python (C11), and who takes both
SELECT st.student_id AS in_statistics, py.student_id AS in_python
FROM (SELECT student_id FROM enrolment WHERE course_id = 'C20') st
FULL OUTER JOIN (SELECT student_id FROM enrolment WHERE course_id = 'C11') py
  ON py.student_id = st.student_id;
```



c = course (left table) · e = enrolment (right table)

course c LEFT JOIN enrolment e

title	student_id	grade
Databases	S01	8.5
Databases	S02	6.5
Databases	S03	5.0
Marketing	*NULL*	*NULL*
Python	S02	9.0
Python	S03	7.5
Statistics	S01	7.0

Marketing has no enrolment:
INNER drops it;
LEFT keeps it, with NULL on the right;
the anti-join keeps only it.

Careful: WHERE e.grade >= 5 removes it again (NULL >= 5 is UNKNOWN). Put that condition in the ON.

Shaded: the rows each join keeps, counted on the sample tables. Only Marketing has no partner.

> **Example.** `SELECT c.title FROM course c LEFT JOIN enrolment e ON e.course_id = c.course_id WHERE e.course_id IS NULL` lists the courses nobody has enrolled in: Marketing.

> **Common mistake.** Filtering the right table in the WHERE: `LEFT JOIN enrolment e ON … WHERE e.grade >= 5` drops the courses without enrolments again, because their grade is NULL. Move the condition into the ON: `ON e.course_id = c.course_id AND e.grade >= 5`.

CROSS JOIN, self-joins, USING and NATURAL

A CROSS JOIN pairs every row with every row of another table. A self-join joins a table with itself under two aliases. USING and NATURAL JOIN are shortcuts for joins on columns with the same name.

- n rows × m rows: a CROSS JOIN grows fast; 1,000 × 1,000 is a million rows.
- Forgetting the join condition of an old-style join (`FROM a, b`) produces an accidental CROSS JOIN.
- Prefer `ON` (or `USING`) to NATURAL JOIN: the join condition is written where everyone can read it.

Example you can run on the website

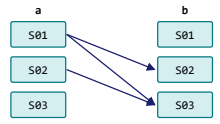
```
SELECT s.name, c.title
FROM student s
CROSS JOIN course c
ORDER BY s.name, c.title;
-- Self-join: pairs of classmates in the same course
SELECT a.course_id, a.student_id AS student_1, b.student_id AS student_2
FROM enrolment a
JOIN enrolment b ON b.course_id = a.course_id AND a.student_id < b.student_id
ORDER BY a.course_id;
SELECT dept_id, d.name AS department, c.title
FROM course c
JOIN department d USING (dept_id);
SELECT title, name FROM course NATURAL JOIN department;
-- A new column with a common name changes the NATURAL JOIN
ALTER TABLE course ADD (name VARCHAR2(30));
SELECT title, name FROM course NATURAL JOIN department;
```

CROSS JOIN: every pair
3 students × 4 courses = 12 rows

	C10	C11	C20	C30
S01	●	○	●	○
S02	●	●	○	○
S03	●	●	○	○

● enrolled · ○ not enrolled:
the CROSS JOIN lists both.

Self-join: enrolment a, enrolment b
Course C10: pairs of classmates



`a.student_id < b.student_id`
3 pairs, each one once;
nobody is paired with themselves.

A CROSS JOIN lists every combination; a self-join pairs rows of one table under two aliases.

> **Example.** Every student with every course: `SELECT s.name, c.title FROM student s CROSS JOIN course c` (3 × 4 = 12 rows).

> **Common mistake.** Relying on NATURAL JOIN in code that has to last. In the example, adding a harmless `name` column to `course` makes `course NATURAL JOIN department` match on `dept_id` **and** `name`, and it silently returns no rows.

Aggregation

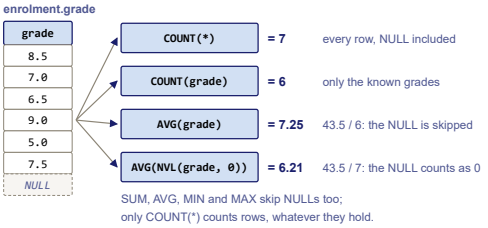
Aggregate functions: COUNT, SUM, AVG, MIN and MAX

An aggregate function turns many rows into one value: how many, the total, the average, the smallest, the largest. Without GROUP BY, all the rows (after the WHERE) form one group and the result is a single row.

- MIN and MAX also work on text (alphabetical order) and dates (earliest, latest); SUM and AVG need numbers.
- On an empty set COUNT returns 0, but SUM, AVG, MIN and MAX return NULL.
- An aggregate and a plain column cannot be mixed without GROUP BY: `SELECT name, COUNT(*) FROM student` fails with ORA-00937 (not a single-group group function).

Example you can run on the website

```
SELECT COUNT(*) AS students, COUNT(email) AS with_email
FROM student;
SELECT COUNT(*) AS enrolments, COUNT(DISTINCT course_id) AS
courses_taken,
       MIN(grade) AS lowest, MAX(grade) AS highest,
ROUND(AVG(grade), 2) AS average
FROM enrolment;
-- A missing grade is ignored, not counted as zero
INSERT INTO enrolment VALUES ('S01', 'C11', NULL);
SELECT COUNT(*) AS enrolments, COUNT(grade) AS graded,
       ROUND(AVG(grade), 2) AS avg_known, ROUND(AVG(NVL(grade,
0)), 2) AS avg_as_zero
FROM enrolment;
```



Aggregates skip NULLs; only COUNT(*) counts every row.

> **Example.** `SELECT COUNT(*) AS enrolments, ROUND(AVG(grade), 2) AS average, MAX(grade) AS best FROM enrolment` sums up the whole table in one row.

> **Common mistake.** Using `COUNT(*)` to count values that may be missing. To know how many students have an email, count the column: `COUNT(email)`.

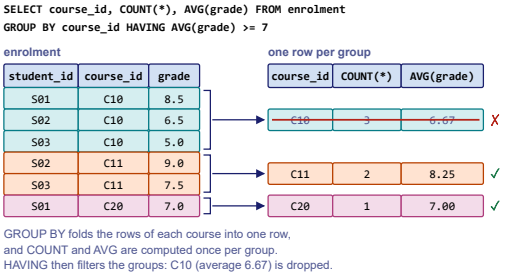
GROUP BY and HAVING

GROUP BY splits the rows into groups that share a value and computes the aggregates once per group. HAVING filters the groups, as WHERE filters the rows.

- WHERE filters rows **before** grouping (`WHERE grade >= 6`: only those grades count); HAVING filters groups **after** it (`HAVING COUNT(*) >= 2`: only groups with two or more rows).
- You can group by several columns: one group per different combination of values.
- A condition without an aggregate belongs in WHERE: it is clearer and discards rows earlier.

Example you can run on the website

```
SELECT course_id, COUNT(*) AS students, ROUND(AVG(grade), 2)
AS average
FROM enrolment
GROUP BY course_id
ORDER BY course_id;
-- Per department: JOIN first, then group by a column of the
joined table
SELECT d.name AS department, COUNT(*) AS courses,
SUM(c.credits) AS credits
FROM course c
JOIN department d ON d.dept_id = c.dept_id
GROUP BY d.name;
-- WHERE (rows) and HAVING (groups) together
SELECT course_id, COUNT(*) AS grades_6_or_more
FROM enrolment
WHERE grade >= 6
GROUP BY course_id
HAVING COUNT(*) >= 2
ORDER BY course_id;
```



One pile per course and one result row per pile; HAVING then throws away whole piles.

Watch the video on the website, in the SQL section.

> **Example.** `SELECT course_id, ROUND(AVG(grade), 2) FROM enrolment GROUP BY course_id HAVING AVG(grade) >= 7` keeps the courses whose average is at least 7.

> **Common mistake.** Writing `WHERE AVG(grade) > 7`, which fails with ORA-00934 (group function is not allowed here). Conditions on aggregates go in HAVING.

Subqueries & sets

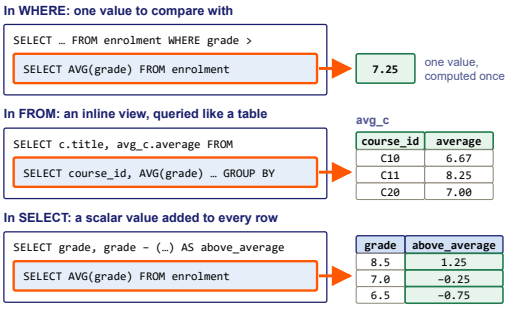
Subqueries in WHERE, FROM and SELECT

A subquery is a SELECT inside another statement, in parentheses. It answers a question the outer query needs first: "what is the average grade?", then "who is above it?".

- If a single-row subquery returns several rows, Oracle fails with ORA-01427 (single-row subquery returns more than one row).
- If it returns no row, its value is NULL, and a comparison with NULL keeps nothing.
- A scalar subquery in SELECT must return at most one row and one column.

Example you can run on the website

```
-- In WHERE: the grades above the overall average
SELECT student_id, course_id, grade
FROM enrolment
WHERE grade > (SELECT AVG(grade) FROM enrolment)
ORDER BY grade DESC;
-- In FROM (inline view): the average per course, then
filtered and joined
SELECT c.title, avg_c.average
FROM (SELECT course_id, ROUND(AVG(grade), 2) AS average
      FROM enrolment
      GROUP BY course_id) avg_c
JOIN course c ON c.course_id = avg_c.course_id
WHERE avg_c.average >= 7;
-- In SELECT (scalar): each grade compared with the overall
average
SELECT student_id, course_id, grade,
       grade - (SELECT AVG(grade) FROM enrolment) AS
above_average
FROM enrolment;
```



The highlighted inner query runs first; where it sits decides what its result is used as.

Watch the video on the website, in the SQL section.

> **Example.** `SELECT title FROM course WHERE credits = (SELECT MAX(credits) FROM course)` lists the courses with the most credits.

> **Common mistake.** Writing `WHERE grade > AVG(grade)`. An aggregate cannot go in WHERE; compute it in a subquery: `WHERE grade > (SELECT AVG(grade) FROM enrolment)`.

Multi-row subqueries: IN, ANY and ALL

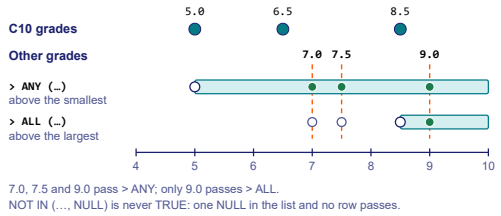
When a subquery returns a list of values, compare with IN (equal to one of them), ANY (true for at least one) or ALL (true for every one). `NOT IN` hides a trap: one NULL in the list and no row is returned.

- A subquery after `=` that returns several rows fails (ORA-01427); use IN.
- `> ALL` with an empty list is true for every row; `> ANY` with an empty list is false.
- IN with a subquery often reads better than a join when you only need the columns of the outer table.

Example you can run on the website

```
SELECT name
FROM student
WHERE student_id IN (SELECT student_id FROM enrolment WHERE course_id = 'C11');
-- Better than every grade of Databases (C10)
SELECT student_id, course_id, grade
```

```
FROM enrolment
WHERE grade > ALL (SELECT grade FROM enrolment WHERE course_id
= 'C10');
-- Better than at least one grade of Databases
SELECT student_id, course_id, grade
FROM enrolment
WHERE course_id <> 'C10'
  AND grade > ANY (SELECT grade FROM enrolment WHERE course_id
= 'C10');
-- The NOT IN trap: the grades nobody got in Python (C11)...
SELECT student_id, course_id, grade
FROM enrolment
WHERE grade NOT IN (SELECT grade FROM enrolment WHERE
course_id = 'C11');
-- ...stop working as soon as one Python grade is NULL
INSERT INTO enrolment VALUES ('S01', 'C11', NULL);
SELECT student_id, course_id, grade
FROM enrolment
WHERE grade NOT IN (SELECT grade FROM enrolment WHERE
course_id = 'C11');
```



`> ANY` means above the smallest value of the list; `> ALL`, above the largest.

Example. The students who take some course of less than 5 credits: `WHERE student_id IN (SELECT e.student_id FROM enrolment e JOIN course c ON c.course_id = e.course_id WHERE c.credits < 5)`.

Common mistake. Using NOT IN with a subquery that can return NULL. In the example, a single Python enrolment without a grade is enough for "the grades nobody got in Python" to return no rows at all.

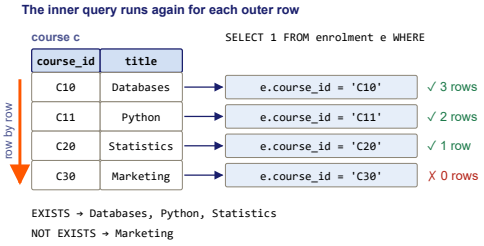
Correlated subqueries and EXISTS

A correlated subquery uses a column of the outer query, so it is evaluated again for each outer row. EXISTS asks whether a subquery returns at least one row; NOT EXISTS, whether it returns none.

- Give the inner table its own alias (`x`), so it is clear which columns belong to the outer row (`e`).
- EXISTS can stop at the first match, and the optimizer often turns EXISTS and IN into joins anyway.
- NOT EXISTS is the safe way to write "has no …"; NOT IN breaks with NULLs.

Example you can run on the website

```
-- Who beat the average of their own course
SELECT e.student_id, e.course_id, e.grade
FROM enrolment e
WHERE e.grade > (SELECT AVG(x.grade) FROM enrolment x WHERE
x.course_id = e.course_id)
ORDER BY e.course_id;
-- Courses with at least one enrolment, and courses with none
SELECT c.title FROM course c
WHERE EXISTS (SELECT 1 FROM enrolment e WHERE e.course_id =
c.course_id);
SELECT c.title FROM course c
WHERE NOT EXISTS (SELECT 1 FROM enrolment e WHERE e.course_id
= c.course_id);
-- Top per group: the best grade of each course
SELECT e.course_id, e.student_id, e.grade
FROM enrolment e
WHERE e.grade = (SELECT MAX(x.grade) FROM enrolment x WHERE
x.course_id = e.course_id)
ORDER BY e.course_id;
```



The inner query runs again with the id of each outer row.

Example. The students not enrolled in Statistics: `SELECT name FROM student s WHERE NOT EXISTS (SELECT 1 FROM enrolment e WHERE e.student_id = s.student_id AND e.course_id = 'C20')`.

Common mistake. Forgetting the correlation condition (`e.course_id = c.course_id`). The subquery then ignores the outer row, and EXISTS is true (or false) for every row at once.

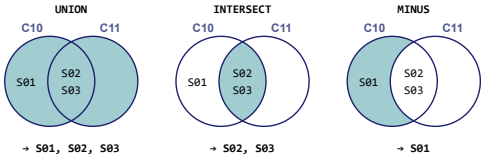
Set operators: UNION, INTERSECT and MINUS

Set operators combine the results of two queries: UNION (rows in either), INTERSECT (rows in both) and MINUS (rows in the first but not in the second). UNION ALL keeps the duplicates.

- Oracle calls the difference **MINUS**; the standard and other DBMSs call it EXCEPT (recent Oracle versions accept both, MINUS works in all of them).
- Set operators compare whole rows, and here two NULLs count as equal.
- To put columns of two tables side by side you need a JOIN; set operators stack rows.

Example you can run on the website

```
-- Students of Databases (C10) and of Python (C11)
SELECT student_id FROM enrolment WHERE course_id = 'C10'
UNION
SELECT student_id FROM enrolment WHERE course_id = 'C11';
SELECT student_id FROM enrolment WHERE course_id = 'C10'
UNION ALL
SELECT student_id FROM enrolment WHERE course_id = 'C11';
SELECT student_id FROM enrolment WHERE course_id = 'C10'
INTERSECT
SELECT student_id FROM enrolment WHERE course_id = 'C11';
SELECT student_id FROM enrolment WHERE course_id = 'C10'
MINUS
SELECT student_id FROM enrolment WHERE course_id = 'C11';
-- One list of names from two tables, sorted once at the end
SELECT name, 'student' AS kind FROM student
UNION
SELECT name, 'department' FROM department
ORDER BY name;
```



C10: students of Databases · C11: students of Python
UNION ALL keeps the repeats: 3 + 2 = 5 rows.

Set operators compare whole result rows: here, the student ids of two courses.

> **Example.** The students of Databases or Statistics: `SELECT student_id FROM enrolment WHERE course_id = 'C10' UNION SELECT student_id FROM enrolment WHERE course_id = 'C20'`.

> **Common mistake.** Writing an ORDER BY inside the first query of a UNION. Oracle fails with ORA-00933 (SQL command not properly ended): sort once, after the last query.

Functions

Text functions

Single-row functions transform one value per row. The text functions change case, cut, search, measure, trim, pad and replace strings; || glues them together.

- Functions can be nested; the innermost runs first: `UPPER(SUBSTR(name, 1, 1))`.
- In Oracle `LENGTH('')` is NULL, not 0, because the empty string is NULL.
- A function on a column in the WHERE can stop an index from being used (see the Efficiency cards).

Example you can run on the website

```
SELECT name,
       UPPER(name) AS shouting,
       LOWER(name) AS quiet,
       LENGTH(name) AS letters,
       SUBSTR(name, 1, INSTR(name, ' ') - 1) AS first_name
FROM student;
-- Split an email into user and domain
SELECT email,
       INITCAP(SUBSTR(email, 1, INSTR(email, '@') - 1)) AS user_part,
       SUBSTR(email, INSTR(email, '@') + 1) AS domain,
       REPLACE(email, 'uni.es', 'cunef.edu') AS new_email
FROM student
WHERE email IS NOT NULL;
-- Pad and glue: a table of contents with dot leaders
SELECT LPAD(SUBSTR(course_id, 2), 4, '0') AS code,
       RPAD(title, 14, '.') || LPAD(credits, 4, '.') AS line
```

```
FROM course;
SELECT '[' || TRIM('   Databases   ') || ']' AS trimmed FROM dual;
```

Example. `SUBSTR(email, INSTR(email, '@') + 1)` returns the domain of an email: `uni.es`.

Common mistake. Counting positions from 0. In Oracle the first character is position 1: `SUBSTR('Oracle', 1, 3)` is `Ora`.

Number and date functions

ROUND, TRUNC, MOD, CEIL and FLOOR shape numbers. SYSDATE, date arithmetic, ADD_MONTHS, MONTHS_BETWEEN and EXTRACT work with dates.

- `ADD_MONTHS(DATE '2026-01-31', 1)` is 28 February: Oracle keeps you at the end of the month.
- `MONTHS_BETWEEN` returns a fraction; wrap it in TRUNC or ROUND for whole months.
- Date literals are written `DATE '2026-09-14'`, always year-month-day.

Example you can run on the website

```
SELECT ROUND(7.456, 2) AS round_2, TRUNC(7.456, 2) AS trunc_2, ROUND(7.5) AS round_0,
       CEIL(4.1) AS up, FLOOR(4.9) AS down, MOD(17, 5) AS remainder
FROM dual;
-- Whole weeks of study at 10 hours a week (1 ECTS credit = 25 hours)
SELECT title, credits * 25 AS hours, CEIL(credits * 25 / 10) AS weeks_at_10h
FROM course;
SELECT DATE '2026-09-14' + 7 AS one_week_later,
       DATE '2027-01-22' - DATE '2026-09-14' AS semester_days,
       ADD_MONTHS(DATE '2026-01-31', 1) AS end_of_february,
       TRUNC(MONTHS_BETWEEN(DATE '2027-06-30', DATE '2026-09-14')) AS whole_months,
       EXTRACT(YEAR FROM DATE '2026-09-14') AS year_part
FROM dual;
SELECT TRUNC(SYSDATE) AS today, TRUNC(SYSDATE, 'MM') AS first_of_month,
       EXTRACT(MONTH FROM SYSDATE) AS month_number
FROM dual;
```

Example. The days left until the end of the semester: `DATE '2027-01-22' - TRUNC(SYSDATE)`.

Common mistake. Adding 30 days to move "one month on". Months have 28 to 31 days: use `ADD_MONTHS`.

Conversion, NULL functions, CASE and DECODE

TO_CHAR, TO_DATE and TO_NUMBER convert between text, dates and numbers with a format model. NVL, NVL2 and COALESCE replace NULLs, and CASE (or Oracle's DECODE) picks a value by conditions.

- To compare dates use `TO_DATE` or a `DATE '…'` literal; never rely on an implicit text-to-date conversion.
- A CASE without ELSE that matches nothing returns NULL.
- Every branch of a CASE must return the same type.

Example you can run on the website

```
SELECT TO_CHAR(TO_DATE('15/09/2026', 'DD/MM/YYYY'), 'YYYY-MM-DD') AS iso_date,
       TO_CHAR(DATE '2026-09-14', 'DD/MM/YYYY') AS spanish_date,
       TO_NUMBER('12.5') * 2 AS doubled,
       TO_CHAR(8.5, 'FM990.00') AS two_decimals
FROM dual;
SELECT name,
       NVL(email, 'no email') AS email,
       NVL2(email, 'yes', 'no') AS has_email,
       COALESCE(email, name, 'unknown') AS contact
FROM student;
SELECT student_id, course_id, grade,
       CASE
         WHEN grade >= 9 THEN 'A'
         WHEN grade >= 7 THEN 'B'
         WHEN grade >= 5 THEN 'C'
         ELSE 'F'
       END AS letter,
```

```
       DECODE(course_id, 'C10', 'Databases', 'C11', 'Python', 'other') AS course_name
FROM enrolment
ORDER BY grade DESC;
```

Example. `CASE WHEN grade >= 5 THEN 'pass' ELSE 'fail' END` turns each grade into a result anyone understands.

Common mistake. Ordering the WHEN clauses badly. `WHEN grade >= 5 THEN 'pass' WHEN grade >= 9 THEN 'distinction'` never says distinction, because the first true branch wins: put the strictest condition first.

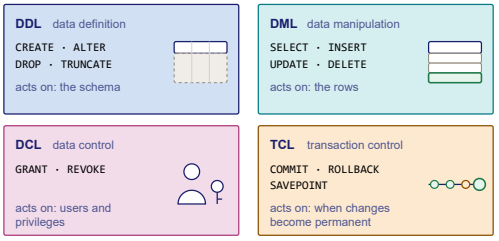
SQL languages

The sub-languages of SQL

SQL is one language with several sub-languages, grouped by what a statement does: define the structure, work with the data, control who can do what, and group changes into transactions.

The four groups

Sub-language	Statements	What it acts on
DDL · data definition	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`	The schema: tables, columns, constraints, indexes, views
DML · data manipulation	`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `MERGE`	The rows stored in the tables
DCL · data control	`GRANT`, `REVOKE`	Users, roles and their privileges
TCL · transaction control	`BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`	When changes become permanent or are undone



Each sub-language acts on something different.

Example. `CREATE TABLE student (…)` is DDL; `INSERT INTO student VALUES (…)` is DML; `GRANT SELECT ON student TO teacher` is DCL; `COMMIT` is TCL.

Common mistake. Thinking `TRUNCATE` is the same as `DELETE` without WHERE. `TRUNCATE` is DDL: it empties the table at once, usually cannot be filtered, and in many DBMSs commits implicitly.

DDL: defining the schema

`CREATE` builds an object, `ALTER` changes it and `DROP` removes it, data included. DDL turns the logical model into real tables.

- Oracle types: `VARCHAR2(n)` for text, `NUMBER(p, s)` for every number, `DATE` (it also stores the time), `TIMESTAMP`, `CLOB`, `BLOB`. From 23ai there is also `BOOLEAN`.
- Auto-numbered keys use `GENERATED ALWAYS AS IDENTITY`; there is no AUTOINCREMENT.
- Each DDL statement updates the data dictionary, which you can query: `USER_TABLES`, `USER_TAB_COLUMNS`, `USER_INDEXES`, `USER_CONSTRAINTS`.

Example you can run on the website

```
CREATE TABLE course (
  course_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title     VARCHAR2(40) NOT NULL,
  credits   NUMBER(3,1) NOT NULL
);
ALTER TABLE course ADD (semester NUMBER(1));
INSERT INTO course (title, credits, semester) VALUES ('Databases', 6, 1);
SELECT * FROM course;
SELECT table_name FROM user_tables;
DROP TABLE course;
SELECT table_name FROM user_tables;
```

Watch the video on the website, in the SQL section.

> **Common mistake.** Dropping and re-creating a table to change one column. `ALTER TABLE` keeps the rows; `DROP` loses them.

CREATE TABLE step by step

A CREATE TABLE gives each column a name, a type and its constraints. A constraint can be written next to its column (inline) or after the last column (out-of-line), where it can cover several columns.

- Name your constraints (`CONSTRAINT fk_player_team …`): error messages and the data dictionary then show a readable name instead of something like `SYS_C008123`.
- A foreign key column has the same type as the key it references.
- In Oracle `DEFAULT` goes before `NOT NULL`: `pos CHAR(1) DEFAULT 'M' NOT NULL`.

Example you can run on the website

```
CREATE TABLE team (
  team_id NUMBER(4)    PRIMARY KEY,
  name    VARCHAR2(40) NOT NULL UNIQUE,
  city    VARCHAR2(30)
);
CREATE TABLE player (
  player_id NUMBER(6)    GENERATED ALWAYS AS IDENTITY,
  full_name VARCHAR2(60) NOT NULL,
  shirt     NUMBER(2)    CHECK (shirt BETWEEN 1 AND 99),
  pos       CHAR(1)      DEFAULT 'M' NOT NULL,
  born      DATE,
  team_id   NUMBER(4)    NOT NULL,
  CONSTRAINT pk_player PRIMARY KEY (player_id),
  CONSTRAINT fk_player_team FOREIGN KEY (team_id) REFERENCES team (team_id),
  CONSTRAINT uq_team_shirt UNIQUE (team_id, shirt),
  CONSTRAINT ck_player_pos CHECK (pos IN ('G', 'D', 'M', 'F'))
);
INSERT INTO team VALUES (1, 'Lions', 'Madrid');
INSERT INTO player (full_name, shirt, pos, born, team_id) VALUES ('Lucia Mora', 9, 'F', DATE '2004-03-12', 1);
INSERT INTO player (full_name, shirt, team_id) VALUES ('Pablo Ruiz', 4, 1);
SELECT * FROM player;
SELECT column_name FROM user_tab_columns WHERE table_name = 'PLAYER' ORDER BY column_id;
INSERT INTO player (full_name, shirt, team_id) VALUES ('Nobody', 7, 99);   -- team 99 does not exist
```

> **Example.** `team` is the parent and `player` the child: every player belongs to an existing team, and a shirt number cannot be repeated within a team.

> **Common mistake.** Creating the child table before its parent. Its FOREIGN KEY fails with ORA-00942 (table or view does not exist), because the referenced table is not there yet.

ALTER, DROP and TRUNCATE

ALTER TABLE changes a table that already holds data: it adds, modifies, renames or drops columns and constraints. DROP TABLE removes the table; TRUNCATE empties it and keeps its structure.

- `ADD`, `RENAME COLUMN` and `DROP COLUMN` run in the browser simulation; `MODIFY`, `ADD CONSTRAINT` and the recycle bin only on FreeSQL (the static script below).
- `ALTER TABLE … ADD CONSTRAINT` checks the existing rows first: it fails if any of them breaks the rule.
- Every ALTER, DROP and TRUNCATE commits the open transaction.

Oracle · run it on FreeSQL

```
DROP TABLE IF EXISTS member PURGE;
CREATE TABLE member (
  member_id NUMBER(4) PRIMARY KEY,
  full_name VARCHAR2(40) NOT NULL,
  email     VARCHAR2(60)
);
INSERT INTO member VALUES (1, 'Ana Ruiz', 'ana@uni.es');
INSERT INTO member VALUES (2, 'Luis Gil', 'luis@uni.es');
```

```
COMMIT;

ALTER TABLE member MODIFY (full_name VARCHAR2(80));         -- make the column longer
ALTER TABLE member MODIFY (email NOT NULL);                  -- works: every row has an email
ALTER TABLE member ADD CONSTRAINT uq_member_email UNIQUE (email);
ALTER TABLE member DROP CONSTRAINT uq_member_email;

DROP TABLE member;                                           -- to the recycle bin
SELECT object_name, original_name FROM user_recyclebin;
FLASHBACK TABLE member TO BEFORE DROP;                       -- back again, rows included
SELECT * FROM member;
DROP TABLE member PURGE;                                     -- gone for good
```

Example you can run on the website

```
CREATE TABLE member (
  member_id NUMBER(4) PRIMARY KEY,
  full_name VARCHAR2(40) NOT NULL
);
INSERT INTO member VALUES (1, 'Ana Ruiz');
INSERT INTO member VALUES (2, 'Luis Gil');
ALTER TABLE member ADD (email VARCHAR2(60), fee NUMBER(5,2) DEFAULT 20 NOT NULL);
ALTER TABLE member RENAME COLUMN full_name TO member_name;
SELECT * FROM member;
ALTER TABLE member DROP COLUMN fee;
-- DELETE can be undone...
DELETE FROM member;
ROLLBACK;
SELECT COUNT(*) AS after_delete_and_rollback FROM member;
-- ...TRUNCATE cannot: it is DDL and commits
TRUNCATE TABLE member;
ROLLBACK;
SELECT COUNT(*) AS after_truncate_and_rollback FROM member;
DROP TABLE member PURGE;
```

Watch the video on the website, in the SQL section.

> **Example.** `ALTER TABLE member ADD (email VARCHAR2(60))` adds a column to a table that already has members: their email is NULL until someone fills it in.

> **Common mistake.** Using TRUNCATE "to try something" on real data. It cannot be rolled back; when in doubt, DELETE and check before you COMMIT.

DML: reading and changing rows

SELECT reads, INSERT adds rows, UPDATE changes them and DELETE removes them. UPDATE and DELETE affect every row that matches the WHERE.

- Joins follow the foreign keys: `enrolment.course_id = course.course_id`.
- The runner shows how many rows each INSERT, UPDATE or DELETE changed.
- To keep only the first rows of a result use `FETCH FIRST n ROWS ONLY`; Oracle has no `LIMIT`.

Example you can run on the website

```
INSERT INTO enrolment VALUES ('S03', 'C20', 6.0);
UPDATE enrolment SET grade = grade + 0.5 WHERE course_id = 'C10' AND grade < 6;
DELETE FROM enrolment WHERE student_id = 'S02' AND course_id = 'C11';
SELECT s.name, c.title, e.grade
FROM enrolment e
JOIN student s ON s.student_id = e.student_id
JOIN course  c ON c.course_id  = e.course_id
ORDER BY s.name, c.title;
SELECT c.title, COUNT(*) AS students, ROUND(AVG(e.grade), 2) AS average
FROM enrolment e JOIN course c ON c.course_id = e.course_id
GROUP BY c.title
HAVING COUNT(*) >= 2;
```

Example. The average grade per course is `SELECT course_id, AVG(grade) FROM enrolment GROUP BY course_id`.

Common mistake. Filtering an aggregate with WHERE (`WHERE AVG(grade) > 7`). WHERE runs before grouping; use `HAVING AVG(grade) > 7`.

INSERT in practice

INSERT adds rows. Name the columns you fill, let DEFAULT and identity columns fill themselves, add several rows in one statement, or copy the rows of a query with INSERT ... SELECT.

- Text and dates go in quotes, numbers without: `'S04'`, `DATE '2026-09-14'`, `6.5`.
- An INSERT that breaks a constraint is rejected whole: with several rows, none of them is inserted.
- The new rows are not permanent until COMMIT.

Example you can run on the website

```
INSERT INTO student (student_id, name) VALUES ('S04', 'Marta Paz');
SELECT * FROM student;
-- Several rows in one statement (Oracle 23ai)
INSERT INTO course (course_id, title, credits, dept_id) VALUES
  ('C40', 'Cloud computing', 6, 'D01'),
  ('C41', 'Ethics', 3, 'D02');
SELECT * FROM course;
-- INSERT ... SELECT: the identity column and the DEFAULT fill themselves
CREATE TABLE honours (
  honours_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  student_id VARCHAR2(3) NOT NULL,
  course_id  VARCHAR2(3) NOT NULL,
  awarded    DATE DEFAULT SYSDATE NOT NULL
);
INSERT INTO honours (student_id, course_id)
SELECT student_id, course_id FROM enrolment WHERE grade >= 8.5;
SELECT * FROM honours;
```

Example. The honours list: `INSERT INTO honours (student_id, course_id) SELECT student_id, course_id FROM enrolment WHERE grade >= 8.5` copies the best results; the id and the date fill themselves.

Common mistake. Leaving out the column list: `INSERT INTO student VALUES ('S04', 'Marta Paz')` fails with ORA-00947 (not enough values), and every INSERT written that way breaks when the table gains a column.

UPDATE and DELETE in practice

UPDATE changes columns of the rows that match its WHERE, with values that can be expressions or subqueries. DELETE removes the matching rows. Before running either one, run its WHERE as a SELECT.

- The runner shows how many rows each statement changed: compare it with the rows the SELECT returned.
- `LEAST(grade + 0.5, 10)` raises a grade without passing the maximum; `GREATEST` does the opposite.
- Until COMMIT, ROLLBACK can still undo a wrong UPDATE or DELETE.

Example you can run on the website

```
-- 1. Run the WHERE as a SELECT: which rows will change?
SELECT * FROM enrolment
WHERE course_id IN (SELECT course_id FROM course WHERE dept_id = 'D01')
  AND grade < 7;
-- 2. The same WHERE in the UPDATE: +0.5, never above 10
UPDATE enrolment
SET grade = LEAST(grade + 0.5, 10)
WHERE course_id IN (SELECT course_id FROM course WHERE dept_id = 'D01')
  AND grade < 7;
-- A value taken from a subquery: Marketing gets as many credits as Databases
UPDATE course
SET credits = (SELECT credits FROM course WHERE course_id = 'C10')
WHERE course_id = 'C30';
-- DELETE with a subquery: the enrolments of every Maths course
```

```
DELETE FROM enrolment
WHERE course_id IN (SELECT course_id FROM course WHERE dept_id = 'D02');
SELECT * FROM enrolment ORDER BY student_id, course_id;
SELECT course_id, title, credits FROM course;
-- Not convinced? Undo it all
ROLLBACK;
```

Example. `UPDATE course SET credits = credits * 1.5 WHERE dept_id = 'D02'` multiplies the credits of every Maths course by 1.5.

Common mistake. Running an UPDATE or DELETE without WHERE, or with a WHERE you never tested. `UPDATE enrolment SET grade = 10` gives everybody a 10.

DCL: users, roles and privileges

GRANT gives a privilege on an object to a user or role; REVOKE takes it away. Give each role only what it needs: the least privilege principle.

- `WITH GRANT OPTION` lets the receiver grant the same privilege to others; use it sparingly.
- REVOKE removes the privilege from that role; members lose it unless another role still grants it.
- Oracle names objects as `schema.table`: `GRANT SELECT ON hr.enrolment TO teacher` refers to the table of the schema `hr`.

Oracle · not runnable here

```
CREATE ROLE teacher;
CREATE ROLE reporting;
CREATE USER ana IDENTIFIED BY "change-me";
GRANT CREATE SESSION TO ana;
GRANT teacher TO ana;

GRANT SELECT, INSERT, UPDATE ON enrolment TO teacher;
GRANT SELECT ON student TO teacher;
GRANT SELECT ON course  TO teacher;

CREATE VIEW course_average AS
  SELECT course_id, AVG(grade) AS average FROM enrolment GROUP BY course_id;
GRANT SELECT ON course_average TO reporting;      -- no access to individual grades

REVOKE UPDATE ON enrolment FROM teacher;
```

Common mistake. Letting the application connect as the database owner. A bug or an SQL injection can then drop tables; with a role limited to DML on its tables, it cannot.

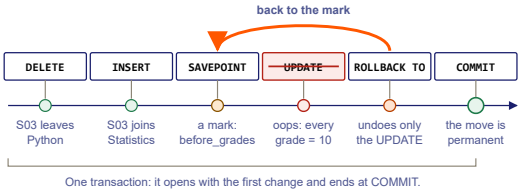
TCL: transactions

A transaction groups statements that must succeed or fail together. COMMIT makes them permanent, ROLLBACK undoes them, and a SAVEPOINT marks a point you can roll back to.

- `SAVEPOINT name` … `ROLLBACK TO SAVEPOINT name` undoes only the work after the savepoint.
- Keep transactions short: while one is open, it may hold locks that make others wait.
- Oracle readers never block writers (it keeps old versions of rows), and by default a query sees the data committed when the query started.

Example you can run on the website

```
DELETE FROM enrolment WHERE student_id = 'S03' AND course_id =
'C11';
INSERT INTO enrolment VALUES ('S03', 'C20', NULL);
SAVEPOINT before_grades;
UPDATE enrolment SET grade = 10;              -- oops: every
row
ROLLBACK TO SAVEPOINT before_grades;
COMMIT;
SELECT * FROM enrolment WHERE student_id = 'S03';
```



ROLLBACK TO SAVEPOINT undoes only the work after the mark; COMMIT makes the rest permanent.

```
DELETE FROM enrolment;
SELECT COUNT(*) AS rows_inside_transaction FROM enrolment;
ROLLBACK;
SELECT COUNT(*) AS rows_after_rollback FROM enrolment;
```

Example. The second block changes a grade by mistake and rolls back to the savepoint: the move between courses is kept, the wrong grade is not.

Common mistake. Running DDL in the middle of a transaction and expecting to roll back. A DDL statement commits everything before it, so the earlier changes can no longer be undone.

Constraints

PRIMARY KEY, UNIQUE and NOT NULL

The primary key identifies each row: unique and never NULL. UNIQUE forbids duplicates in other candidate keys, and NOT NULL makes a column mandatory.

- Composite key: `PRIMARY KEY (student_id, course_id)`.
- The last statement of the example fails on purpose: read the error message.

Example you can run on the website

```
INSERT INTO student VALUES ('S04', 'Marta Paz', NULL);   -- a second NULL e-mail is allowed
SELECT * FROM student;
INSERT INTO student VALUES ('S05', 'Ana Ruiz', 'ana@uni.es');  -- duplicate e-mail
```

Watch the video on the website, in the SQL section.

Common mistake. Using only a surrogate id and forgetting the natural key. With `id` as PK and no `UNIQUE (email)`, the same student can be inserted twice.

FOREIGN KEY and referential actions

A foreign key only accepts values that exist in the referenced key. Its referential action says what happens to child rows when the parent row is deleted or its key changes.

- CASCADE fits parts that make no sense alone (an enrolment without its student, an order line without its order).
- No action (the default) protects reference data (a course, a department).
- Inserting a child with a missing parent fails with ORA-02291 (parent key not found); deleting a parent with children fails with ORA-02292 (child record found).

Example you can run on the website

```
DELETE FROM student WHERE student_id = 'S01';     -- CASCADE: Ana's enrolments go too
SELECT * FROM enrolment ORDER BY student_id;
DELETE FROM course WHERE course_id = 'C10';       -- C10 still has enrolments: rejected
```

Common mistake. Using CASCADE everywhere. Deleting one department could then silently delete its courses and all their enrolments.

CHECK and DEFAULT

CHECK states a condition each row must meet, such as a grade between 0 and 10. DEFAULT gives a column its value when the INSERT does not say one.

- A CHECK passes when the condition is true **or unknown** (NULL); add NOT NULL if the value is required.

Example you can run on the website

```
CREATE TABLE loan (
  loan_id   NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  book      VARCHAR2(60) NOT NULL,
  loan_date DATE DEFAULT SYSDATE NOT NULL,
  days      NUMBER(2) DEFAULT 15 NOT NULL,
```

```
  status    VARCHAR2(10) DEFAULT 'open' NOT NULL,
  CONSTRAINT days_range CHECK (days BETWEEN 1 AND 60),
  CONSTRAINT status_values CHECK (status IN ('open', 'returned', 'lost'))
);
INSERT INTO loan (book) VALUES ('Don Quixote');
INSERT INTO loan (book, days, status) VALUES ('Ficciones', 30, 'returned');
SELECT * FROM loan;
INSERT INTO loan (book, days) VALUES ('Hopscotch', 90);       -- breaks days_range
```

> **Common mistake.** Checking ranges only in the application. Data loaded with a script or another program then skips the rule.

Views & schema objects

Views

A view is a stored query with a name. You query it like a table, but it stores the query, not the rows: every time you read it, the query runs again on the current data.

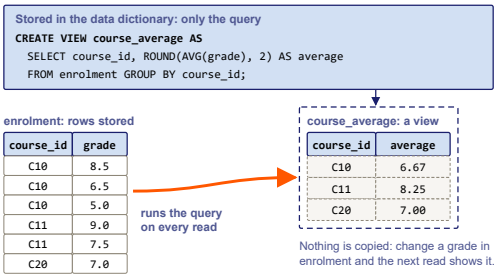
- A view with a join, GROUP BY or DISTINCT is normally not updatable directly.
- `DROP VIEW name` removes the view; its tables and rows are untouched.
- A **materialized view** does store rows and is refreshed from time to time: a photograph of the room, not a window.

Oracle · run it on FreeSQL

```
DROP TABLE IF EXISTS product PURGE;
CREATE TABLE product (
  product_id NUMBER(4) PRIMARY KEY,
  name       VARCHAR2(40) NOT NULL,
  price      NUMBER(6,2) NOT NULL,
  on_sale    CHAR(1) DEFAULT 'N' NOT NULL
);
INSERT INTO product VALUES (1, 'Notebook', 3.50, 'Y');
INSERT INTO product VALUES (2, 'Pen', 1.20, 'N');
COMMIT;

-- An updatable view that only lets you work on the products
on sale
CREATE OR REPLACE VIEW sale_items AS
  SELECT product_id, name, price, on_sale FROM product WHERE
on_sale = 'Y'
  WITH CHECK OPTION;
UPDATE sale_items SET price = 3.00 WHERE product_id = 1;    --
changes the table
INSERT INTO sale_items VALUES (3, 'Ruler', 2.10, 'Y');       -
- visible through the view: accepted
INSERT INTO sale_items VALUES (4, 'Glue', 1.80, 'N');        -
- ORA-01402: view WITH CHECK OPTION where-clause violation
SELECT * FROM product;

-- A view nobody can write through
CREATE OR REPLACE VIEW price_list AS
  SELECT name, price FROM product
  WITH READ ONLY;
UPDATE price_list SET price = 0;                             -
- ORA-42399: cannot perform a DML operation on a read-only
view
```



A view is a window: it stores the query, and the rows come from the base table at every read.

Example you can run on the website

```
CREATE OR REPLACE VIEW student_grades AS
SELECT s.name, c.title, e.grade
FROM enrolment e
JOIN student s ON s.student_id = e.student_id
JOIN course  c ON c.course_id  = e.course_id;
SELECT * FROM student_grades WHERE grade >= 7 ORDER BY grade DESC;
```

```
-- The view stores the query, not the rows: change a grade in the table...
UPDATE enrolment SET grade = 9.5 WHERE student_id = 'S03' AND course_id = 'C10';
-- ...and the view shows it at once
SELECT * FROM student_grades WHERE name = 'Eva Sanz';
CREATE OR REPLACE VIEW course_average AS
SELECT course_id, COUNT(*) AS students, ROUND(AVG(grade), 2) AS average
FROM enrolment
GROUP BY course_id;
SELECT * FROM course_average;
```

Example. `CREATE OR REPLACE VIEW course_average AS SELECT course_id, ROUND(AVG(grade), 2) AS average FROM enrolment GROUP BY course_id` gives everybody the averages without showing a single grade.

Common mistake. Thinking a view is a copy that must be refreshed. A normal view always shows the current data, and it is exactly as fast as the query inside it.

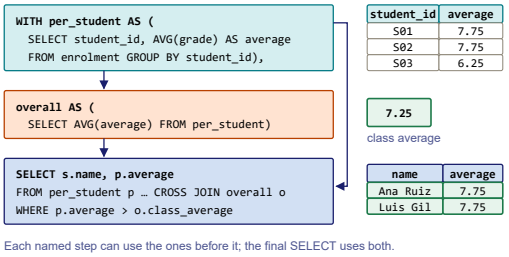
WITH: common table expressions

The WITH clause gives a name to a subquery at the start of a statement, a common table expression (CTE), and the main query uses it like a table. Several CTEs can be chained, each one using the ones before it.

- One WITH per statement, at the start; the CTEs are separated by commas and the main SELECT comes last.
- A CTE can be used several times in the main query, instead of repeating the same subquery.
- A CTE that refers to itself (a recursive CTE) walks hierarchies such as an organisation chart.

Example you can run on the website

```
WITH course_avg AS (
  SELECT course_id, ROUND(AVG(grade), 2) AS average
  FROM enrolment
  GROUP BY course_id
)
SELECT c.title, ca.average
FROM course_avg ca
JOIN course c ON c.course_id = ca.course_id
ORDER BY ca.average DESC;
-- Chained CTEs: each step uses the previous one
WITH per_student AS (
  SELECT student_id, AVG(grade) AS average
  FROM enrolment
  GROUP BY student_id
),
overall AS (
  SELECT AVG(average) AS class_average FROM per_student
)
SELECT s.name, ROUND(p.average, 2) AS average,
ROUND(o.class_average, 2) AS class_average
FROM per_student p
JOIN student s ON s.student_id = p.student_id
CROSS JOIN overall o
WHERE p.average > o.class_average;
```



Each CTE is a named step; the later steps and the final SELECT use it like a table.

Example. The students whose average is above the class average: one CTE with the average of each student, another with the overall average, and a final SELECT that compares them.

Common mistake. Writing a semicolon or a second WITH between the CTEs. They are one statement: `WITH a AS (…), b AS (…) SELECT …`.

Sequences and synonyms

A sequence hands out unique numbers, one after another, to any session that asks. A synonym is another name for an object, often for a table of another schema.

- Sequences can leave **gaps** (rollbacks, cached numbers lost when the database restarts): never use them to count rows.
- `CACHE 20` keeps numbers in memory for speed; `NOCYCLE` (the default) stops at MAXVALUE instead of starting again.

- The browser simulation cannot run sequences or synonyms: run the script on FreeSQL.

Sequence or identity column?

	Sequence	Identity column
Belongs to	The schema: any table can use it	One column of one table
How to use it	`seq.NEXTVAL` in the INSERT, or as the column DEFAULT	Leave the column out of the INSERT
Shared numbering	Yes	No
Gaps	Possible	Possible (it uses a sequence inside)

Oracle · run it on FreeSQL

```
DROP TABLE IF EXISTS ticket PURGE;
DROP SEQUENCE IF EXISTS ticket_seq;
CREATE SEQUENCE ticket_seq START WITH 100 INCREMENT BY 1 NOCACHE;
CREATE TABLE ticket (
  ticket_no NUMBER DEFAULT ticket_seq.NEXTVAL PRIMARY KEY,
  customer  VARCHAR2(30) NOT NULL
);
INSERT INTO ticket (customer) VALUES ('Ana');                                    -- 100, from the DEFAULT
INSERT INTO ticket (ticket_no, customer) VALUES (ticket_seq.NEXTVAL, 'Luis');     -- 101
SELECT ticket_seq.CURRVAL AS last_ticket FROM dual;                              -- 101 again, in this session
SELECT * FROM ticket;

-- A synonym: a nickname for the table
CREATE OR REPLACE SYNONYM queue FOR ticket;
SELECT * FROM queue;
DROP SYNONYM queue;
```

> **Common mistake.** Expecting a sequence without gaps, such as invoice numbers 1, 2, 3… with none missing. A rolled-back INSERT still used its number; if the law requires consecutive numbers, generate them another way.

PL/SQL

PL/SQL blocks, variables and SELECT INTO

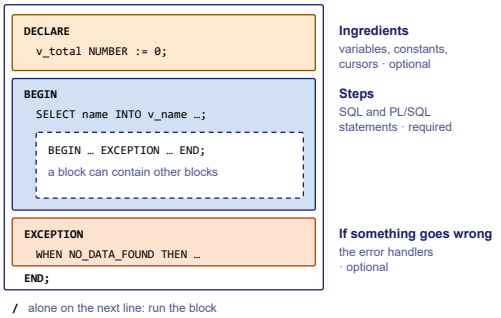
PL/SQL is Oracle's procedural language: SQL plus variables, conditions, loops and error handling. Code is written in blocks: an optional DECLARE section, a BEGIN … END body and an optional EXCEPTION section.

- `:=` assigns and `=` compares; every statement ends with `;`.
- Prefix the names (`v_` variable, `c_` constant, `r_` record) so they never clash with column names.
- If SELECT INTO finds no row it raises NO_DATA_FOUND; if it finds several, TOO_MANY_ROWS (see the exceptions card).

Oracle PL/SQL · run it on FreeSQL

```
DROP TABLE IF EXISTS enrolment CASCADE CONSTRAINTS PURGE;
DROP TABLE IF EXISTS student CASCADE CONSTRAINTS PURGE;
CREATE TABLE student (
  student_id VARCHAR2(3) PRIMARY KEY,
  name       VARCHAR2(40) NOT NULL
);
CREATE TABLE enrolment (
  student_id VARCHAR2(3) REFERENCES student (student_id),
  course_id  VARCHAR2(3),
  grade      NUMBER(4,2),
  PRIMARY KEY (student_id, course_id)
);
INSERT INTO student VALUES ('S01', 'Ana Ruiz');
INSERT INTO enrolment VALUES ('S01', 'C10', 8.5);
INSERT INTO enrolment VALUES ('S01', 'C20', 7.0);
COMMIT;

DECLARE
  c_pass    CONSTANT NUMBER := 5;
  r_student student%ROWTYPE;              -- a whole row of
```



The parts of a block. Only BEGIN … END is required.

```
student
  v_average enrolment.grade%TYPE;         -- the type of the gra
  v_courses PLS_INTEGER;
BEGIN
  SELECT * INTO r_student FROM student WHERE student_id = 'S01';
  SELECT AVG(grade), COUNT(*) INTO v_average, v_courses
  FROM enrolment
  WHERE student_id = r_student.student_id;

  DBMS_OUTPUT.PUT_LINE(r_student.name || ' takes ' || v_courses
  DBMS_OUTPUT.PUT_LINE('Average: ' || TO_CHAR(v_average, 'FM90.0
  IF v_average >= c_pass THEN
    DBMS_OUTPUT.PUT_LINE('On track to pass.');
  END IF;
END;
/
```

Example. The block of the example reads one student and their average into variables and prints a sentence about them, which a single SELECT cannot do.

Common mistake. Giving a variable the name of a column (`name`). In `WHERE name = name` both sides mean the column, so the condition is true for every row. Call it `v_name`.

Control flow: IF, CASE and loops

PL/SQL decides with IF … ELSIF … ELSE and CASE, and repeats with three loops: the basic LOOP with EXIT WHEN, WHILE, and the numeric FOR.

- Every loop needs a way out: a basic LOOP without EXIT runs forever.
- `CONTINUE WHEN condition;` skips to the next lap.
- In real programs the loop usually walks through rows: that is what cursors are for (next card).

Oracle PL/SQL · run it on FreeSQL

```
DECLARE
  v_grade   NUMBER := 7.5;
  v_label   VARCHAR2(20);
  v_balance NUMBER := 1000;
  v_years   PLS_INTEGER := 0;
BEGIN
  -- IF / ELSIF / ELSE: the first true branch wins
  IF v_grade >= 9 THEN
    v_label := 'Outstanding';
  ELSIF v_grade >= 7 THEN
    v_label := 'Good';
  ELSIF v_grade >= 5 THEN
    v_label := 'Pass';
  ELSE
    v_label := 'Fail';
  END IF;
  DBMS_OUTPUT.PUT_LINE(v_grade || ' is ' || v_label);

  -- CASE statement on a value
  CASE v_label
    WHEN 'Fail' THEN DBMS_OUTPUT.PUT_LINE('See you at the resit.');
    WHEN 'Pass' THEN DBMS_OUTPUT.PUT_LINE('Just made it.');
    ELSE DBMS_OUTPUT.PUT_LINE('Well done!');
  END CASE;

  -- FOR: a counted loop
  FOR i IN 1 .. 5 LOOP
    DBMS_OUTPUT.PUT_LINE('7 x ' || i || ' = ' || 7 * i);
  END LOOP;

  -- WHILE: years until 1,000 euros at 5% a year reach 1,500
  WHILE v_balance < 1500 LOOP
    v_balance := v_balance * 1.05;
```

```
    v_years := v_years + 1;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(v_years || ' years: ' || ROUND(v_balance, 2) || ' euros');

  -- LOOP ... EXIT WHEN: a countdown
  LOOP
    DBMS_OUTPUT.PUT_LINE('Countdown: ' || v_years);
    v_years := v_years - 3;
    EXIT WHEN v_years <= 0;
  END LOOP;
END;
/
```

Example. Grading a mark: `IF v_grade >= 9 THEN … ELSIF v_grade >= 7 THEN … ELSIF v_grade >= 5 THEN … ELSE … END IF;`.

Common mistake. Writing `ELSE IF` or `ELSEIF`. PL/SQL wants `ELSIF`; `ELSE IF` opens a new IF that needs its own `END IF;`.

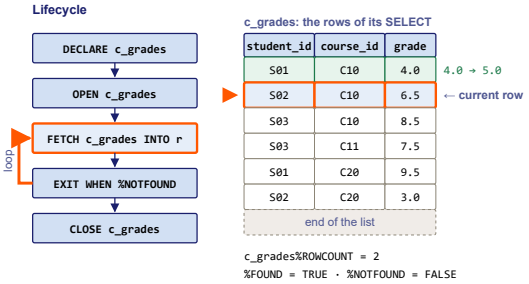
Cursors: walking through the rows one by one

A cursor points at the rows of a query so PL/SQL can process them one at a time. Oracle opens an implicit cursor for every SQL statement; you declare an explicit cursor when you need to visit the rows of a SELECT one by one.

- After any INSERT, UPDATE or DELETE, the implicit cursor tells you how many rows changed (`SQL%ROWCOUNT`) and whether any did (`SQL%FOUND`).
- `FOR UPDATE` locks the rows until COMMIT or ROLLBACK, so nobody changes them halfway through; commit after closing the cursor, not inside the loop.
- In a cursor FOR loop the record `r` is declared for you, and the cursor is always closed, even after an error.

Cursor attributes

Attribute	Means	Typical use
`c%ISOPEN`	The cursor is open	`IF NOT c%ISOPEN THEN OPEN c; END IF;`
`c%FOUND`	The last FETCH returned a row	Process the row
`c%NOTFOUND`	The last FETCH found nothing: the end of the list	`EXIT WHEN c%NOTFOUND;`
`c%ROWCOUNT`	Rows fetched so far	Number the lines of a report
`SQL%ROWCOUNT`	Rows changed by the last SQL statement (implicit cursor)	`DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' rows')`



A cursor FOR loop does the OPEN, FETCH, EXIT and CLOSE for you.

The cursor is a finger on the rows of its SELECT: each FETCH moves it down one row.

Oracle PL/SQL · run it on FreeSQL

```
DROP TABLE IF EXISTS enrolment CASCADE CONSTRAINTS PURGE;
CREATE TABLE enrolment (
  student_id VARCHAR2(3),
  course_id  VARCHAR2(3),
  grade      NUMBER(4,2) CHECK (grade BETWEEN 0 AND 10),
  PRIMARY KEY (student_id, course_id)
);
INSERT INTO enrolment VALUES ('S01', 'C10', 4.0);
INSERT INTO enrolment VALUES ('S02', 'C10', 6.5);
INSERT INTO enrolment VALUES ('S03', 'C10', 8.5);
INSERT INTO enrolment VALUES ('S03', 'C11', 7.5);
INSERT INTO enrolment VALUES ('S01', 'C20', 9.5);
INSERT INTO enrolment VALUES ('S02', 'C20', 3.0);
COMMIT;
```

```
-- 1. Explicit cursor: DECLARE -> OPEN -> FETCH in a loop -> CLOSE
DECLARE
  CURSOR c_grades IS
    SELECT student_id, course_id, grade
    FROM enrolment
    ORDER BY course_id, student_id
    FOR UPDATE OF grade;
  r     c_grades%ROWTYPE;
  v_new enrolment.grade%TYPE;
BEGIN
  OPEN c_grades;
  LOOP
    FETCH c_grades INTO r;
    EXIT WHEN c_grades%NOTFOUND;              -- the finger reached the end of the list
    v_new := CASE
               WHEN r.course_id = 'C10' AND r.grade < 5 THEN r.grade + 1
               WHEN r.course_id = 'C10' AND r.grade < 7 THEN r.grade + 0.5
               WHEN r.course_id = 'C20'                 THEN r.grade + 1
               ELSE r.grade
             END;
    v_new := LEAST(v_new, 10);                -- never above 10
    IF v_new <> r.grade THEN
      UPDATE enrolment SET grade = v_new WHERE CURRENT OF c_grades;
      DBMS_OUTPUT.PUT_LINE(c_grades%ROWCOUNT || '. ' || r.student_id || ' ' || r.course_id || ': ' || r.grade || '
-> ' || v_new);
    END IF;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(c_grades%ROWCOUNT || ' rows read');
  CLOSE c_grades;
  COMMIT;
END;
/

-- 2. The cursor FOR loop: OPEN, FETCH, EXIT and CLOSE are automatic
DECLARE
  CURSOR c_list IS
    SELECT student_id, course_id, grade FROM enrolment ORDER BY course_id, student_id;
BEGIN
  FOR r IN c_list LOOP
    DBMS_OUTPUT.PUT_LINE(r.course_id || ' ' || r.student_id || ': ' || r.grade);
  END LOOP;
  -- The implicit cursor of a plain UPDATE
  UPDATE enrolment SET grade = LEAST(grade + 0.25, 10) WHERE course_id = 'C11';
  DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' Python grade(s) raised');
  COMMIT;
END;
/

-- 3. The curve of block 1 as ONE set-based statement (prefer it when a formula is enough):
-- UPDATE enrolment
-- SET grade = LEAST(10, grade + CASE
--                                WHEN course_id = 'C10' AND grade < 5 THEN 1
--                                WHEN course_id = 'C10' AND grade < 7 THEN 0.5
--                                WHEN course_id = 'C20' THEN 1
--                                ELSE 0
--                              END);
```

Watch the video on the website, in the SQL section.

> **Example.** S01's 4 in Databases becomes 5, S02's 6.5 becomes 7 and the 8.5 stays; in Statistics everyone gains a point, but the 9.5 stops at 10.

> **Common mistake.** Writing a cursor loop when **one set-based UPDATE** would do. If each row only needs a formula on its own values, `UPDATE enrolment SET grade = LEAST(10, grade + CASE … END)` changes the whole table in one statement, much faster. Keep cursors for work SQL cannot express in one statement: several steps per row, calls to procedures, messages or decisions that depend on earlier rows.

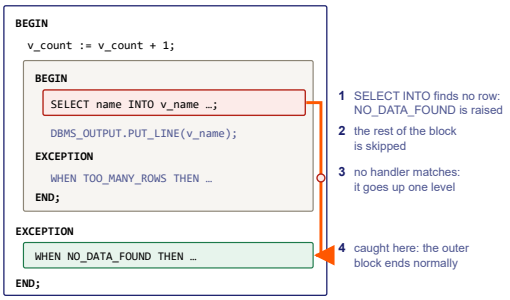
Exceptions: handling errors

When a statement fails inside PL/SQL, Oracle raises an exception and jumps to the EXCEPTION section of the block. There you catch errors by name and deal with them, and the program carries on instead of stopping.

- Put the specific handlers first and WHEN OTHERS last.
- An exception that a block does not handle passes to the enclosing block, and in the end to the caller.
- Inside a loop, a nested block handles the error of one row and the loop continues with the next.

Common predefined exceptions

Exception	Error	Raised when
NO_DATA_FOUND	ORA-01403	A SELECT INTO returns no row
TOO_MANY_ROWS	ORA-01422	A SELECT INTO returns more than one row
ZERO_DIVIDE	ORA-01476	A number is divided by zero
DUP_VAL_ON_INDEX	ORA-00001	An INSERT or UPDATE repeats a primary key or UNIQUE value
VALUE_ERROR	ORA-06502	A value does not fit its variable (too long, wrong type)
INVALID_NUMBER	ORA-01722	Text that is not a number is used as one in SQL



If no block handles it, the error reaches the caller and the script stops.

An exception jumps to the nearest handler that matches it; if none does, it climbs to the enclosing block.

Oracle PL/SQL · run it on FreeSQL

```
DROP TABLE IF EXISTS student CASCADE CONSTRAINTS PURGE;
CREATE TABLE student (
  student_id VARCHAR2(3) PRIMARY KEY,
  name       VARCHAR2(40) NOT NULL,
  credits    NUMBER(3) DEFAULT 0 NOT NULL
);
INSERT INTO student VALUES ('S01', 'Ana Ruiz', 60);
INSERT INTO student VALUES ('S02', 'Luis Gil', 0);
COMMIT;

DECLARE
  v_name    student.name%TYPE;
  v_credits student.credits%TYPE;
  e_too_many_credits EXCEPTION;               -- a user-defined exception
BEGIN
  BEGIN                                       -- NO_DATA_FOUND
    SELECT name INTO v_name FROM student WHERE student_id = 'S99';
  EXCEPTION
    WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('No student S99');
  END;

  BEGIN                                       -- TOO_MANY_ROWS
    SELECT name INTO v_name FROM student;
  EXCEPTION
    WHEN TOO_MANY_ROWS THEN DBMS_OUTPUT.PUT_LINE('More than one student: use a cursor');
  END;

  BEGIN                                       -- ZERO_DIVIDE: Luis has 0 credits
    SELECT credits INTO v_credits FROM student WHERE student_id = 'S02';
    DBMS_OUTPUT.PUT_LINE('Hours per credit: ' || 150 / v_credits);
  EXCEPTION
    WHEN ZERO_DIVIDE THEN DBMS_OUTPUT.PUT_LINE('Cannot divide by zero credits');
  END;

  BEGIN                                       -- DUP_VAL_ON_INDEX: S01 already exists
    INSERT INTO student (student_id, name) VALUES ('S01', 'Ana again');
  EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN DBMS_OUTPUT.PUT_LINE('S01 already exists');
  END;
```

```
  BEGIN                                       -- a business rule with a user exception
    SELECT credits INTO v_credits FROM student WHERE student_id = 'S01';
    IF v_credits + 30 > 78 THEN
      RAISE e_too_many_credits;
    END IF;
  EXCEPTION
    WHEN e_too_many_credits THEN DBMS_OUTPUT.PUT_LINE('No more than 78 credits a year');
  END;

  BEGIN                                       -- your own error number, caught by WHEN OTHERS
    RAISE_APPLICATION_ERROR(-20001, 'Enrolment is closed');
  EXCEPTION
    WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Error ' || SQLCODE || ': ' || SQLERRM);
  END;
END;
/
```

Example. Looking up a student who does not exist prints "No student S99", and the script carries on with the next step.

Common mistake. Writing `WHEN OTHERS THEN NULL;`. It swallows every error in silence: the program seems to work and the data is wrong. At least log SQLERRM, or re-raise the error with `RAISE;`.

Stored procedures and functions

A procedure is a named PL/SQL block, stored in the database, that does something. A function returns a value and can be used inside SQL. Both take parameters.

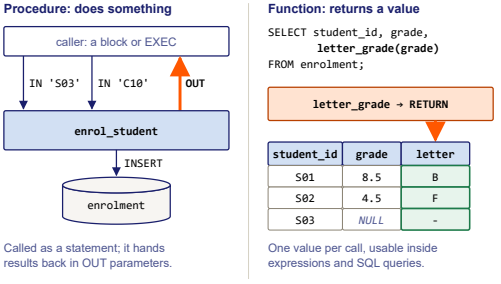
- If it compiles with errors, Oracle stores it as INVALID; `SHOW ERRORS` (or the `USER_ERRORS` view) says why.
- A function called from SQL should not change data.
- Prefix parameters with `p_` so they never clash with column names.

Oracle PL/SQL · run it on FreeSQL

```
DROP TABLE IF EXISTS enrolment CASCADE CONSTRAINTS PURGE;
CREATE TABLE enrolment (
  student_id VARCHAR2(3),
  course_id  VARCHAR2(3),
  grade      NUMBER(4,2),
  PRIMARY KEY (student_id, course_id)
);
INSERT INTO enrolment VALUES ('S01', 'C10', 8.5);
INSERT INTO enrolment VALUES ('S02', 'C10', 4.5);
COMMIT;

-- A function: a calculator that returns a value
CREATE OR REPLACE FUNCTION letter_grade (p_grade IN NUMBER)
RETURN VARCHAR2 IS
BEGIN
  RETURN CASE
           WHEN p_grade IS NULL THEN '-'
           WHEN p_grade >= 9 THEN 'A'
           WHEN p_grade >= 7 THEN 'B'
           WHEN p_grade >= 5 THEN 'C'
           ELSE 'F'
         END;
END letter_grade;
/

-- A procedure: a vending machine that does something and
hands back a message
CREATE OR REPLACE PROCEDURE enrol_student (
  p_student IN  VARCHAR2,
  p_course  IN  VARCHAR2,
  p_message OUT VARCHAR2
) IS
BEGIN
  INSERT INTO enrolment (student_id, course_id) VALUES
```



A procedure is called as a statement; a function returns a value and can be used in SQL.

```
(p_student, p_course);
  p_message := p_student || ' enrolled in ' || p_course;
EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    p_message := p_student || ' was already enrolled in ' || p_c
END enrol_student;
/

-- Calling both from a block...
DECLARE
  v_msg VARCHAR2(100);
BEGIN
  enrol_student('S03', 'C10', v_msg);
  DBMS_OUTPUT.PUT_LINE(v_msg);
  enrol_student('S01', 'C10', v_msg);
  DBMS_OUTPUT.PUT_LINE(v_msg);
  DBMS_OUTPUT.PUT_LINE('8.5 is a ' || letter_grade(8.5));
  COMMIT;
END;
/

-- ...and the function from SQL
SELECT student_id, grade, letter_grade(grade) AS letter
FROM enrolment
ORDER BY student_id;
```

Watch the video on the website, in the SQL section.

Example. `letter_grade(8.5)` returns `B`, and `SELECT student_id, grade, letter_grade(grade) FROM enrolment` applies it to every row.

Common mistake. Writing a procedure where a function is needed, or the opposite. If you want a value to use in an expression or a query, write a function; if you want an action, a procedure.

Triggers

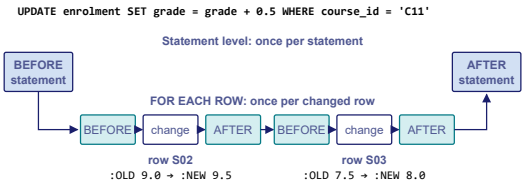
A trigger is PL/SQL that Oracle runs automatically when something happens to a table: before or after an INSERT, UPDATE or DELETE, once per statement or once for each row.

- In an INSERT the `:OLD` values are NULL; in a DELETE the `:NEW` values are.
- Typical uses are audit trails, derived values and rules across tables; simple rules belong in constraints.
- A trigger runs inside the transaction of its statement: if the statement is rolled back, so is the trigger's work.

Oracle PL/SQL · run it on FreeSQL

```
DROP TABLE IF EXISTS grade_audit PURGE;
DROP TABLE IF EXISTS enrolment CASCADE CONSTRAINTS PURGE;
CREATE TABLE enrolment (
  student_id VARCHAR2(3),
  course_id  VARCHAR2(3),
  grade      NUMBER(4,2),
  PRIMARY KEY (student_id, course_id)
);
CREATE TABLE grade_audit (
  audit_id   NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  student_id VARCHAR2(3),
  course_id  VARCHAR2(3),
  old_grade  NUMBER(4,2),
  new_grade  NUMBER(4,2),
  changed_by VARCHAR2(128) DEFAULT USER,
  changed_at DATE DEFAULT SYSDATE
);
INSERT INTO enrolment VALUES ('S01', 'C10', 4.5);
INSERT INTO enrolment VALUES ('S02', 'C10', 6.0);
COMMIT;

-- BEFORE ... FOR EACH ROW: check and adjust the new value
```



BEFORE row can check or change :NEW; AFTER row sees the final values, for example to write a line in grade_audit.

Firing order for one UPDATE that changes two rows.

```
CREATE OR REPLACE TRIGGER trg_grade_check
BEFORE INSERT OR UPDATE OF grade ON enrolment
FOR EACH ROW
BEGIN
  IF :NEW.grade > 10 THEN
    RAISE_APPLICATION_ERROR(-20010, 'A grade cannot be above 10'
  END IF;
  :NEW.grade := ROUND(:NEW.grade, 1);          -- keep one decim
END;
/

-- AFTER ... FOR EACH ROW with WHEN: audit only real changes
CREATE OR REPLACE TRIGGER trg_grade_audit
AFTER UPDATE OF grade ON enrolment
FOR EACH ROW
WHEN (NEW.grade <> OLD.grade OR (OLD.grade IS NULL AND NEW.grade
BEGIN
  INSERT INTO grade_audit (student_id, course_id, old_grade, new
  VALUES (:OLD.student_id, :OLD.course_id, :OLD.grade, :NEW.grad
END;
/

-- AFTER, statement level: once, however many rows changed
CREATE OR REPLACE TRIGGER trg_grade_notice
AFTER UPDATE OF grade ON enrolment
BEGIN
  DBMS_OUTPUT.PUT_LINE('Grades updated at ' || TO_CHAR(SYSDATE,
END;
/

UPDATE enrolment SET grade = grade + 0.54 WHERE course_id = 'C10
COMMIT;
SELECT * FROM enrolment;
SELECT student_id, old_grade, new_grade, changed_by, TO_CHAR(cha
FROM grade_audit;
```

Watch the video on the website, in the SQL section.

> **Example.** Every change of a grade writes a line in `grade_audit` with who, when, the old grade and the new one, without the application doing anything.

> **Common mistake.** Hiding business logic in triggers, or querying the same table from a row-level trigger. Chains of invisible actions are hard to debug, and reading the table being changed fails with ORA-04091 (table is mutating). Keep triggers small: auditing and checks that constraints cannot express.

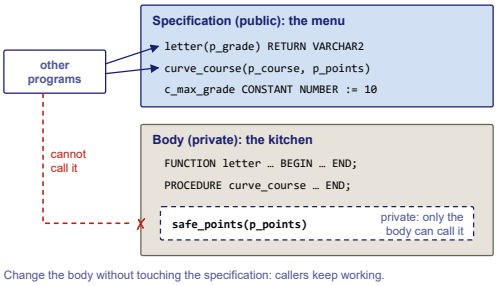
Packages

A package groups related procedures, functions, constants and cursors under one name. It has two parts: the specification, its public interface, and the body, the hidden implementation.

- Changing only the body does not invalidate the programs that call the package.
- Two subprograms of a package can share a name if their parameters differ (overloading).
- Grant EXECUTE on the package, not on each procedure.

Oracle PL/SQL · run it on FreeSQL

```
DROP TABLE IF EXISTS enrolment CASCADE CONSTRAINTS PURGE;
CREATE TABLE enrolment (
  student_id VARCHAR2(3),
  course_id  VARCHAR2(3),
  grade      NUMBER(4,2),
  PRIMARY KEY (student_id, course_id)
);
INSERT INTO enrolment VALUES ('S01', 'C10', 4.5);
INSERT INTO enrolment VALUES ('S02', 'C10', 7.25);
COMMIT;
```



Change the body without touching the specification: callers keep working.

Callers only see the specification; the body and its private helpers stay hidden.

```
-- Specification: the menu (public)
CREATE OR REPLACE PACKAGE grades_pkg IS
  c_max_grade CONSTANT NUMBER := 10;
  FUNCTION  letter (p_grade IN NUMBER) RETURN VARCHAR2;
  PROCEDURE curve_course (p_course IN VARCHAR2, p_points IN NUMB
END grades_pkg;
/

-- Body: the kitchen (private details)
CREATE OR REPLACE PACKAGE BODY grades_pkg IS
  -- Private: not in the specification, so only this body can us
  FUNCTION safe_points (p_points IN NUMBER) RETURN NUMBER IS
  BEGIN
    RETURN LEAST(GREATEST(p_points, 0), 2);      -- a curve is b
  END safe_points;

  FUNCTION letter (p_grade IN NUMBER) RETURN VARCHAR2 IS
  BEGIN
    RETURN CASE WHEN p_grade >= 9 THEN 'A' WHEN p_grade >= 7 THE
                WHEN p_grade >= 5 THEN 'C' ELSE 'F' END;
  END letter;

  PROCEDURE curve_course (p_course IN VARCHAR2, p_points IN NUMB
    v_points NUMBER := safe_points(p_points);
  BEGIN
    UPDATE enrolment
    SET grade = LEAST(grade + v_points, c_max_grade)
    WHERE course_id = p_course;
    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' grades curved by ' ||
  END curve_course;
END grades_pkg;
/

BEGIN
  grades_pkg.curve_course('C10', 5);           -- asks for 5; th
  COMMIT;
END;
/
SELECT student_id, grade, grades_pkg.letter(grade) AS letter FRO
```

> **Example.** `grades_pkg` publishes `letter` and `curve_course`; the rule that a curve is between 0 and 2 points is a private function that nobody outside the package can call or skip.

> **Common mistake.** Putting every helper in the specification. Whatever is public becomes a promise: other code starts using it and you can no longer change it freely. Publish only what callers need.

Indexes

What an index is

An index is a separate, sorted structure (usually a B+ tree) that maps the values of some columns to their rows. With it, the DBMS can search instead of scanning the whole table.

- The primary key and each UNIQUE constraint already have an index.
- Foreign keys do **not** get one automatically in Oracle; index them if you join or filter by them.

Example you can run on the website

```
EXPLAIN PLAN FOR SELECT * FROM customer WHERE email = 'user43210@mail.com';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT * FROM customer WHERE email = 'user43210@mail.com';
CREATE INDEX idx_customer_email ON customer (email);
EXPLAIN PLAN FOR SELECT * FROM customer WHERE email = 'user43210@mail.com';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT * FROM customer WHERE email = 'user43210@mail.com';
```

Watch the video on the website, in the SQL section.

> **Common mistake.** Thinking an index changes the result. It only changes how fast the DBMS finds the rows; the query returns exactly the same data.

Composite and covering indexes

A composite index sorts by several columns, in order. It helps queries that filter on its leftmost columns. A covering index contains every column a query needs, so the table is not read at all.

- Leftmost-prefix rule: `(a, b, c)` serves filters on `a`, `a, b` and `a, b, c`.
- One composite index can replace several single-column ones.

Example you can run on the website

```
CREATE INDEX idx_city_date ON customer (city, signup_date);
EXPLAIN PLAN FOR SELECT * FROM customer WHERE city = 'Madrid' AND signup_date >= DATE '2023-01-01';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
EXPLAIN PLAN FOR SELECT * FROM customer WHERE signup_date >= DATE '2023-01-01';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
EXPLAIN PLAN FOR SELECT city, signup_date FROM customer WHERE city = 'Sevilla';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT COUNT(*) AS madrid_since_2023 FROM customer WHERE city = 'Madrid' AND signup_date >= DATE '2023-01-01';
```

> **Common mistake.** Creating `(signup_date, city)` for queries that always fix the city and give a date range. With the range first, the second column cannot narrow the search.

What indexes cost

Indexes speed up reads but slow down writes: every INSERT, UPDATE or DELETE must also update each index of the table. They also take disk and memory.

When to index a column

Usually yes	Usually no
Foreign keys used in joins	Columns never used in WHERE, JOIN or ORDER BY
Columns filtered with = or ranges that return few rows	Low-selectivity columns (gender, yes/no)
Columns used to sort large results (ORDER BY … LIMIT)	Small tables that fit in a few blocks
Search keys of frequent queries	Tables with heavy writes and rare reads (logs)

Example you can run on the website

```
INSERT INTO plain_log
SELECT LEVEL, MOD(LEVEL, 997), 'action' || MOD(LEVEL, 13), DATE '2024-01-01' + MOD(LEVEL, 365)
FROM dual CONNECT BY LEVEL <= 20000;
INSERT INTO indexed_log
SELECT LEVEL, MOD(LEVEL, 997), 'action' || MOD(LEVEL, 13), DATE '2024-01-01' + MOD(LEVEL, 365)
FROM dual CONNECT BY LEVEL <= 20000;
SELECT index_name AS indexes_on_indexed_log FROM user_indexes WHERE table_name = 'INDEXED_LOG';
```

> **Common mistake.** Indexing every column "just in case". Each index slows every write and the optimizer still uses only the useful ones.

Kinds of index

The B-tree is the default and serves equality, ranges and sorting. Other kinds solve specific problems: hash for equality only, bitmap for few distinct values, GIN for values inside a document or array, BRIN for huge, naturally ordered tables.

Main index kinds

Kind	Good for	Not for
B-tree / B+ tree	=, <, >, BETWEEN, ORDER BY, prefix LIKE 'abc%'	Searching inside text or arrays
Hash	Equality only (=) (PostgreSQL, MySQL MEMORY)	Ranges and sorting

Kind	Good for	Not for
Bitmap	Columns with few distinct values in read-mostly tables (data warehouses)	Tables with many concurrent writes
Reverse key (Oracle)	Monotonic keys (sequence numbers) inserted by many sessions at once: spreads the hot right-hand leaf	Range scans: it destroys the key order
GIN / inverted	Full-text search, JSON keys, array elements	Simple scalar columns
BRIN	Very large tables ordered by insertion (dates in a log) (PostgreSQL)	Randomly ordered data

Oracle · not runnable here

```
CREATE INDEX orders_customer_idx ON orders (customer_id);               -- B-tree (default)
CREATE BITMAP INDEX customer_city_bix ON customer (city);               -- few distinct values, data warehouse
CREATE INDEX orders_id_rev ON orders (order_id) REVERSE;                -- reverse-key
CREATE INDEX customer_email_ci ON customer (LOWER(email));              -- function-based
CREATE UNIQUE INDEX open_order_uq ON orders (CASE WHEN status = 'OPEN' THEN customer_id END);  -- "partial" unique
```

> **Common mistake.** Creating a bitmap index on a table that many sessions update at once. Each change locks a whole range of rows in the bitmap, so writers wait for each other.

Clustering

Clustered and non-clustered indexes

In a clustered index the table rows themselves are stored in the order of the key, inside the index. A table can have only one. Every other index is non-clustered: it stores the key plus a pointer to the row.

- Choose a clustering key that is short, does not change, and matches frequent range queries.
- Random keys (such as random UUIDs) scatter inserts all over a clustered index; increasing keys append at the end.

Example you can run on the website

```
CREATE TABLE enrolment_by_course (
  course_id  VARCHAR2(3),
  student_id VARCHAR2(3),
  grade      NUMBER(4,2),
  PRIMARY KEY (course_id, student_id)
) ORGANIZATION INDEX;
INSERT INTO enrolment_by_course VALUES ('C20', 'S01', 7.0);
INSERT INTO enrolment_by_course VALUES ('C10', 'S03', 5.0);
INSERT INTO enrolment_by_course VALUES ('C10', 'S01', 8.5);
INSERT INTO enrolment_by_course VALUES ('C11', 'S02', 9.0);
INSERT INTO enrolment_by_course VALUES ('C10', 'S02', 6.5);
SELECT * FROM enrolment_by_course;           -- stored in key order (without ORDER BY the order is never
guaranteed)
EXPLAIN PLAN FOR SELECT * FROM enrolment_by_course WHERE course_id = 'C10';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

> **Common mistake.** Confusing a **clustered index** with a **database cluster**. The second is a group of servers working together (replication, high availability), an unrelated meaning of the word.

Table clusters in Oracle (and CLUSTER in PostgreSQL)

An Oracle table cluster stores the rows of several tables that share a key (a department and its employees) in the same blocks, which speeds up joins on that key. PostgreSQL has no such thing: its `CLUSTER` command rewrites a table once, in the order of an index.

Oracle · not runnable here

```
CREATE CLUSTER dept_cluster (dept_id VARCHAR2(3));
CREATE INDEX dept_cluster_idx ON CLUSTER dept_cluster;      -- a cluster needs its index

CREATE TABLE department (
  dept_id VARCHAR2(3) PRIMARY KEY,
  name    VARCHAR2(30) NOT NULL
```

```
) CLUSTER dept_cluster (dept_id);

CREATE TABLE course (
  course_id VARCHAR2(3) PRIMARY KEY,
  title     VARCHAR2(40) NOT NULL,
  dept_id   VARCHAR2(3) NOT NULL REFERENCES department (dept_id)
) CLUSTER dept_cluster (dept_id);                          -- same blocks as its department
```

> **Common mistake.** Clustering tables that are updated all the time or often read alone. Clusters only pay off for stable data that is read together.

Partitioning

Horizontal partitioning

Partitioning splits one big table into smaller pieces, the partitions, according to a partition key. Queries still use the single table name; the DBMS sends each row to its partition.

Choosing the method

Method	Typical key	Use it when
Range	Date, id ranges	Queries and data retention go by time
List	Country, region, status	A few known values separate the data
Hash	Customer id	You need even sizes and there is no natural range

Oracle · not runnable here

```
CREATE TABLE orders (
  order_id    NUMBER(10),
  customer_id NUMBER(10) NOT NULL,
  order_date  DATE NOT NULL,
  total       NUMBER(10,2) NOT NULL
)
PARTITION BY RANGE (order_date) (
  PARTITION p2025_01 VALUES LESS THAN (DATE '2025-02-01'),
  PARTITION p2025_02 VALUES LESS THAN (DATE '2025-03-01'),
  PARTITION p_max    VALUES LESS THAN (MAXVALUE)             -- rows outside every range
);

-- Oracle creates the monthly partitions by itself:
-- PARTITION BY RANGE (order_date) INTERVAL (NUMTOYMINTERVAL(1, 'MONTH')) (PARTITION p_first VALUES LESS THAN
(DATE '2025-01-01'))

CREATE INDEX orders_customer_ix ON orders (customer_id) LOCAL;   -- one index segment per partition
```

> **Common mistake.** Partitioning a small table. Below many millions of rows, a good index is usually simpler and just as fast.

Partition pruning and maintenance

When the WHERE fixes the partition key, the optimizer reads only the matching partitions: partition pruning. Old data can then be removed by dropping a whole partition instead of deleting millions of rows.

- Choose the partition key from the most frequent filters.
- A **local** index has one segment per partition and is kept valid when you drop a partition; a unique index must include the partition key to be local.

Oracle · not runnable here

```
EXPLAIN PLAN FOR SELECT SUM(total) FROM orders
WHERE order_date >= DATE '2025-02-01' AND order_date < DATE '2025-03-01';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
-- | Id | Operation              | Name   | Pstart | Pstop |
-- |  2 |  PARTITION RANGE SINGLE|        |      2 |     2 |    <- only one partition is read
-- |  3 |   TABLE ACCESS FULL    | ORDERS |      2 |     2 |
```

```
ALTER TABLE orders DROP PARTITION p2025_01 UPDATE INDEXES;   -- remove a whole month at once
ALTER TABLE orders EXCHANGE PARTITION p2025_01 WITH TABLE orders_2025_01_archive;  -- …or swap it out as a table
```

> **Common mistake.** Partitioning by a column the queries rarely filter on. Without pruning, every query visits every partition.

Vertical partitioning and sharding

Vertical partitioning splits the columns of a table into two tables with the same key. Sharding splits the rows across different servers, each with its own database.

Three ways to split

Technique	Splits	Where the pieces live	Main cost
Horizontal partitioning	Rows	Same database	Queries without the key visit every partition
Vertical partitioning	Columns	Same database	A join to read the whole row
Sharding	Rows	Different servers	Cross-shard joins and transactions

> **Common mistake.** Sharding before it is needed. Indexes, partitioning, read replicas and a bigger server solve most problems with far less complexity.

Efficiency

Reading a query plan

SQL says what you want; the optimizer decides how: which index, which join order, which algorithm. EXPLAIN PLAN shows that plan, and the plan is where slow queries are understood.

- TABLE ACCESS FULL = read every row; INDEX RANGE SCAN / INDEX UNIQUE SCAN = go down an index.
- SORT ORDER BY = an extra sort; an index in that order avoids it.

Example you can run on the website

```
EXPLAIN PLAN FOR
SELECT c.city, SUM(o.total) FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT c.city, ROUND(SUM(o.total), 2) AS spent FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
CREATE INDEX idx_orders_customer ON orders (customer_id);
EXEC DBMS_STATS.GATHER_TABLE_STATS(USER, 'ORDERS');
EXPLAIN PLAN FOR
SELECT c.city, SUM(o.total) FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT c.city, ROUND(SUM(o.total), 2) AS spent FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
```

> **Common mistake.** Optimizing by intuition. Read the plan first: the slow part is often not where you expect.

Conditions that can use an index

An index on a column only helps when the condition compares the bare column. Wrapping it in a function or calculation hides it from the index: the query scans.

Example you can run on the website

```
EXPLAIN PLAN FOR SELECT COUNT(*) FROM customer WHERE TO_CHAR(signup_date, 'YYYY') = '2023';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
EXPLAIN PLAN FOR SELECT COUNT(*) FROM customer WHERE signup_date >= DATE '2023-01-01' AND signup_date < DATE
'2024-01-01';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
EXPLAIN PLAN FOR SELECT * FROM customer WHERE LOWER(email) = 'user77@mail.com';
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
CREATE INDEX idx_email_lower ON customer (LOWER(email));
EXPLAIN PLAN FOR SELECT * FROM customer WHERE LOWER(email) = 'user77@mail.com';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

> **Common mistake.** Adding an index and assuming the query will use it. Check the plan; a function on the column is enough to make the index useless.

Writing efficient queries

Ask the database for exactly what you need, in one set-based statement, and let it do the work near the data: filter, join and aggregate in SQL, not row by row in the application.

- Select only the columns you use; `SELECT *` reads and sends more data and stops covering indexes from helping.
- Use `EXISTS` to ask "is there any?" instead of `COUNT(*) > 0`: it can stop at the first match.
- Paginate with `ORDER BY … FETCH FIRST n ROWS ONLY` (`OFFSET m ROWS` to skip), ideally on an indexed column.
- Join and filter on indexed keys, and keep transactions short.

Example you can run on the website

```
-- "Has customer 42 ordered anything?": EXISTS can stop at the first order
SELECT CASE WHEN EXISTS (SELECT 1 FROM orders WHERE customer_id = 42) THEN 'yes' ELSE 'no' END AS has_orders FROM
dual;
-- One set-based statement instead of a loop over every order
UPDATE orders SET total = ROUND(total * 0.9, 2) WHERE order_date < DATE '2023-02-01';
COMMIT;
-- Only the columns and rows needed, aggregated in the database
SELECT c.city, COUNT(*) AS orders, ROUND(SUM(o.total), 2) AS spent
FROM orders o JOIN customer c ON c.customer_id = o.customer_id
WHERE o.order_date BETWEEN DATE '2024-01-01' AND DATE '2024-12-31'
GROUP BY c.city
ORDER BY spent DESC
FETCH FIRST 5 ROWS ONLY;
```

> **Example.** The last query of the example answers "top 5 cities by spending in 2024" with one statement, instead of reading every order into the program.

> **Common mistake.** Fetching a whole table to filter it in the application. The database can use indexes and send only the matching rows; the program can do neither.